REVIN

PLANT SIMULATOR

Full-Stack Project Documentation

v1.0 · 2025

BUILT WITH

React 18 · Node/Express · PostgreSQL · Prisma ORM · Chart.js

Table of Contents

1. Overview & Domain Model

RvinEngine is a full-stack Plant Growth Simulator. Users define **conditions** (variables such as water or light) with optional **importance weights**, then run a **preview simulation** that maps each condition to Low / Normal / High and models **health**, **stress**, and **growth** over 1–365 days. Results are displayed as a Chart.js line chart alongside final-day scores.

1.1 The Variable (Condition) Model

Conditions are persisted in PostgreSQL via Prisma as **Variable** records. Each record carries the following fields:

Field	Type	Role
`key`	String (slug)	Stable unique identifier used in API payloads and simulation
`name`	String	Human-readable label shown in the UI
`description`	String?	Optional notes about the condition
`weight`	Float (default 0.1)	Importance multiplier in the simulation preview
`sortOrder`	Int	Display order — lower values appear first

*Keys are auto-derived from the name when omitted: lowercased, non-alphanumeric characters replaced with underscores, and trimmed. Attempting to create a duplicate key returns **409 Conflict**.*

2. User Flows

2.1 Conditions Page (/variables)

- Loads all variables via **GET /api/variables**, ordered by sortOrder then name.
- Allows creating new variables via **POST /api/variables** with optional key, description, weight, and sort order.

2.2 Preview Growth (/ or /simulator)

- Loads the same variable list.
- For each variable, the user selects **Normal** (default), **Low**, or **High**. An empty selection maps to Normal.
- User sets the number of **days** (1–365).
- Submits **POST /api/simulations/run** with the days count and a key→level map.
- Results are rendered as a line chart (health, stress, growth vs day) plus end-of-run scores.

3. Simulation Logic & Engine

3.1 Request Handling (Server)

1	Validate days — must be an integer in the range 1–365.
2	Validate variables — must be a plain object.
3	Load all Variable records from the database (key, weight).
4	For each DB variable: read the level from the request body; missing or invalid values default to medium. Build a weights map from each row's weight field (fallback 0.1).
5	Call runSimulation({ days, variables, weights }) and return JSON 201.

Every defined condition is always included in the simulation — conditions missing from the request body default to medium rather than being ignored.

3.2 Core Engine Rules

The engine (**simulationEngine.js**) is a **deterministic model with small random jitter** per day — not a scientific crop model.

Level Scores

Level	Score	Meaning
`low`	–1	Unfavourable condition
`medium`	0	Neutral / baseline
`high`	+1	Favourable condition
`(invalid)`	0	Treated as medium

Weighted Balance (per day)

For each simulation day, a *conditionBalance* is computed as the sum of each condition's level score multiplied by its weight:

```
conditionBalance = Σ  levelScore(level) × weight   (for all condition keys)
```

Initial State & Clamping

Metric	Start	Clamp Range
`health`	80	[0, 100]
`stress`	20	[0, 100]
`growth`	5	[0, 100]

Daily Update Logic

- If conditionBalance < 0 → stress increases (scaled by |conditionBalance|).

• If conditionBalance > 0 → growth receives a positive boost.

• Base growth drift, stress coupling, health coupling, and high-stress penalties (stress > 70 and stress > 85 thresholds) adjust all three metrics.

• Small random noise is added to each metric.

• All values are clamped to [0, 100] after each day's update.

Engine Output

Field	Description
`history`	Array of { day, health, stress, growth } for every simulated day
`summary`	Final-day values: finalHealth, finalStress, finalGrowth
`input`	Echoes resolved days, normalised variables map, and weights used

4. System Architecture

RvinEngine is a classic three-tier web application. The browser hosts a React/Vite SPA; an Express server handles API requests and runs the simulation engine; PostgreSQL (via Prisma ORM) stores conditions.

Layer Overview

Layer	Technology	Responsibility
Browser (SPA)	React 18 + Vite 6 + Tailwind 4	UI rendering, routing, Chart.js visualisation
HTTP Client	Axios	REST calls to Express API via VITE_API_BASE_URL
API Server	Node.js + Express	Route handling, request validation, business logic
Simulation Engine	Pure JS function	Stateless day-by-day growth model
ORM / DB	Prisma + @prisma/adapter-pg	Schema management, typed queries, connection pooling
Database	PostgreSQL	Persistent storage for Variable records

Request Flow

A simulation run follows this path through the system:

1. SimulatorPage.jsx → api.js (runSimulation)
2. Axios POST /api/simulations/run
3. Express router → simulationsController
4. Controller loads Variables from Prisma / PostgreSQL
5. Controller calls runSimulation() in simulationEngine.js
6. JSON 201 response → axios → React state → Chart.js

5. Repository Layout

```
RvinEngine/
■■■ backend/
■   ■■■ prisma/
■   ■   ■■■ schema.prisma          # Variable model
■   ■■■ prisma-client/            # Generated Prisma client
■   ■■■ src/
■       ■■■ server.js             # Express app entry point
■       ■■■ lib/prisma.js         # PrismaClient + pg pool
■       ■■■ routes/               # Thin HTTP routers
■       ■■■ controllers/          # Validation + DB + engine calls
■       ■■■ services/
■           ■■■ simulationEngine.js
■■■ frontend/
■   ■■■ src/
■       ■■■ main.jsx              # React root + BrowserRouter
■       ■■■ App.jsx               # Shell: sidebar nav + routes
■       ■■■ pages/
■       ■   ■■■ VariablesPage.jsx  # Conditions CRUD UI
■       ■   ■■■ SimulatorPage.jsx  # Preview form + chart
■       ■■■ services/
■           ■■■ axiosClient.js    # Axios base URL config
■           ■■■ api.js            # REST endpoint wrappers
■■■ present.ps1                   # Windows: start both servers
```

6. Backend Reference

Module	Responsibility
`server.js`	Loads .env; configures CORS and JSON parsing; mounts /api/variables, /api/simulations; exposes GET /api/health liveness check
`variablesRoutes.js`	GET / → list conditions; POST / → create condition
`simulationsRoutes.js`	POST /run → execute simulation
`variablesController.js`	Input validation, key slugification, Prisma create/findMany, 409 conflict handling
`simulationsController.js`	Request validation, variable loading, level normalisation, engine invocation
`lib/prisma.js`	Single PrismaClient instance using @prisma/adapter-pg and pg pool; requires DATABASE_URL env var
`simulationEngine.js`	Pure function runSimulation({ days, variables, weights }); returns { history, summary, input }

7. Frontend Reference

Module	Responsibility
`axiosClient.js`	Axios instance; baseURL set from VITE_API_BASE_URL (must include /api)
`api.js`	Thin wrappers: getVariables(), createVariable(), runSimulation()
`App.jsx`	Layout shell: brand bar, sidebar nav, React Router routes (/ and /simulator → SimulatorPage, /variables → VariablesPage)
`VariablesPage.jsx`	Lists all conditions; create-condition form with key, name, description, weight, sortOrder fields
`SimulatorPage.jsx`	Fetches variables; renders days input + per-variable level selects; calls runSimulation; displays summary cards and Chart.js Line chart of history data

8. API Quick Reference

Method	Path	Description	Status
GET	/api/health	Liveness check	200
GET	/api/variables	List all conditions, ordered by sortOrder then name	200
POST	/api/variables	Create condition — body: name (req), key, description, weight, sortOrder	201 / 409
POST	/api/simulations/run	Run simulation — body: days (int 1–365), variables (key→level map)	201

Run Response Shape

```
{
  "input": {
    "days": 30,
    "variables": { "water": "high", "light": "low" },
    "weights":   { "water": 0.8,   "light": 0.5  }
  },
  "summary": {
    "finalHealth": 74.2,
    "finalStress": 31.6,
    "finalGrowth": 18.9
  },
  "history": [
    { "day": 1, "health": 80, "stress": 20, "growth": 5.3 },
    ...
  ]
}
```

9. Configuration & Running

Environment Variables

Variable	Side	Description
PORT	Backend	HTTP port (default 3000)
DATABASE_URL	Backend	PostgreSQL connection string — required by Prisma
VITE_API_BASE_URL	Frontend	Full base URL including /api, e.g. http://localhost:3000/api

Starting the Project

On Windows the included **present.ps1** PowerShell script starts both servers in separate console windows and opens the Vite dev server (default **http://localhost:5173**) in your browser.

```
# Backend
cd backend && npm run dev      # starts Express on PORT (default 3000)

# Frontend (separate terminal)
cd frontend && npm run dev     # starts Vite on http://localhost:5173

# Or use the PowerShell helper (Windows)
.\present.ps1
```

Database Setup

```
# Apply schema to the database
cd backend && npx prisma db push

# Or use migrations
npx prisma migrate dev --name init

# Regenerate Prisma client after schema changes
npx prisma generate
```

10. Design Decisions

Server-side simulation	The engine lives entirely on the server so the growth model stays in one authoritative place. Clients only send levels and a day count — they never need to implement or version the simulation rules.
DB-backed weights	Condition importance weights are stored in PostgreSQL, not hardcoded. This means 'importance' is editable at runtime without a code deployment.
Routes vs Controllers	HTTP wiring (method, path, middleware) is kept in thin router files, separate from validation and business logic in controller files. The engine itself is a pure function, making it independently testable and reusable.
Deterministic + jitter	The engine produces slightly different results on each run due to small random noise, making the simulation feel organic while remaining fast and predictable in its overall trajectory.
Default to medium	Any condition key not present in the simulation request (or with an unrecognised level string) defaults to 'medium' (score 0). This ensures every defined condition always participates in the simulation.